# PQC-LINK: Progress Report 3

The handshake benchmark between two computers

RSCL senior design team

October 19, 2026

**Abstract**

Reports 1 and 2 measured the secure link on one computer, over loopback. This report moves the T10 handshake benchmark onto *two* computers on a local network: a native Linux server and a client in a Windows WSL2 virtual machine. It compares configuration **A** (classical `X25519`) with configuration **B** (hybrid `X25519MLKEM768`) and measures the cost on *both* ends. Three valid sessions of 100 handshakes per configuration agree: the hybrid handshake costs about 0.140 to 0.231 ms more than the classical one, and about 1.2 times the processor time on each side. Absolute times move by a quarter between sessions, so the ratios are the stable result. The hybrid configuration also showed two connection attempts per session that took about one second to connect; we did not investigate the cause. Two earlier sessions were invalid and are kept but not used. These are development computers, one of them a virtual machine, so none of this is gate evidence: the Raspberry Pi measurements are still to come.

## 1 What we did

The benchmark programs already existed (Reports 1 and 2). To run them across a network we added two scripts, one per machine, a configuration file, and a runbook so that anyone on the team can repeat it.

| New | What it is |
|---|---|
| `bench_server_remote.sh` | Runs the **A** and **B** servers on the server machine, on the LAN address; writes the server's handshake files and an environment record. Refuses to start if its ports are in use. |
| `bench_client_remote.sh` | Runs the benchmark client against them; writes the client's files and an environment record, including the commit and the round-trip time. |
| `t10_handshake_lan.conf` | The experiment: both configurations interleaved, 100 measured handshakes each after 5 warm-up ones. |
| `two_machine_setup.md` | Runbook: certificates, firewall, smoke test, the three sessions, and the mistakes we made on the way. |

**Table 1:** What was added for the two-computer run.

Before the benchmark we ran the link itself between the two machines with both configurations: 20 records each delivered and acknowledged, the same records accepted again as duplicates, and three refusals that must fail (wrong certificate authority, wrong host name, mismatched configuration). All passed.

## 2 The setup

The server is the native Linux machine and the client is the WSL2 machine. This direction is deliberate: a WSL2 machine sits behind network address translation, so it can *make* connections to the LAN but other machines cannot easily reach it. Putting the client there needs no inbound setup.

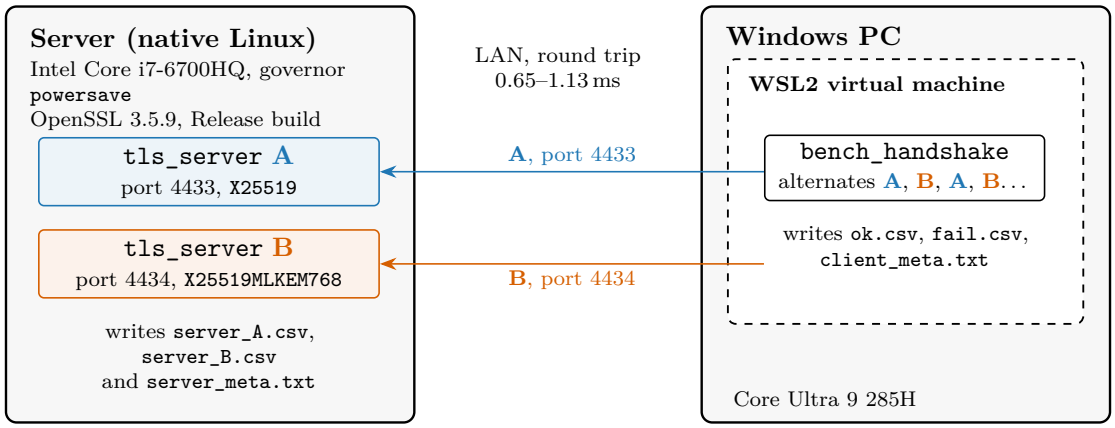



**Figure 1:** The two machines. The client connects out to the server (solid arrows). The only files that travel between them are the certificate authority's public certificate (once, by `scp`, fingerprint compared on both sides) and, after a session, the server's result files. Private keys never leave the server.

| Item | Value |
|---|---|
| Server | native Linux, Intel(R) Core(TM) i7-6700HQ CPU @ 2.60GHz, CPU governor `powersave` |
| Client | Intel(R) Core(TM) Ultra 9 285H under WSL2 (a virtual machine; no governor is visible, `NA`) |
| Network | one LAN, no loss or delay added; `ping` round trips (min/avg/max/deviation): 0.661/0.822/1.127/0.165 ms (session 1), 0.695/0.807/0.946/0.094 ms (2), 0.646/0.845/1.031/0.148 ms (3) |
| Software | OpenSSL 3.5.9, Release build, both ends; session 1 commit `8af4295`, sessions 2 and 3 `aa14646` / `aa14646`; working tree clean, same commit on both machines (yes/yes/yes) |
| TLS | 1.3 only, `TLS_AES_128_GCM_SHA256`, group pinned on both ends; ECDSA P-256 demo certificates |
| Experiment | 100 measured handshakes per configuration per session after 5 warm-up, **A** and **B** interleaved, a new TCP connection and a full handshake every time (no resumption) |
| Date (UTC) | 2026-10-20, all three sessions |

**Table 2:** Conditions. They are recorded by the scripts next to every run, not typed afterwards.

## 3 How a session runs, and when it counts

Each session uses a new run identifier, the same on both machines, so results never overwrite each other. The order matters: the server must be started first and must be the only one.

The server records its own view of every handshake, so a session is a cross-check of two independent records. This is what caught the two invalid sessions (Section 6).

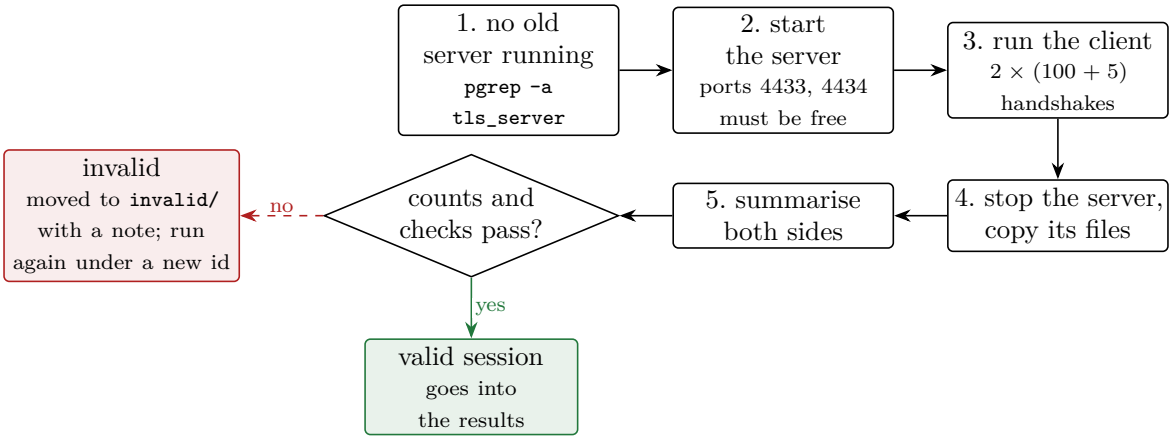



**Figure 2:** One benchmark session. Step 5 is the check: a session is valid only if each configuration shows 100 measured handshakes and no failures on both the client and the server side, the byte counts agree (what the client sent is what the server received), and the summary prints no warning.

# 4 Results

## 4.1 Handshake time

Figure 3 shows the time of the TLS handshake alone, measured by the client. Every session has the same shape: **B** sits above **A** by a small, stable amount, and the spread of each is a few tenths of a millisecond.

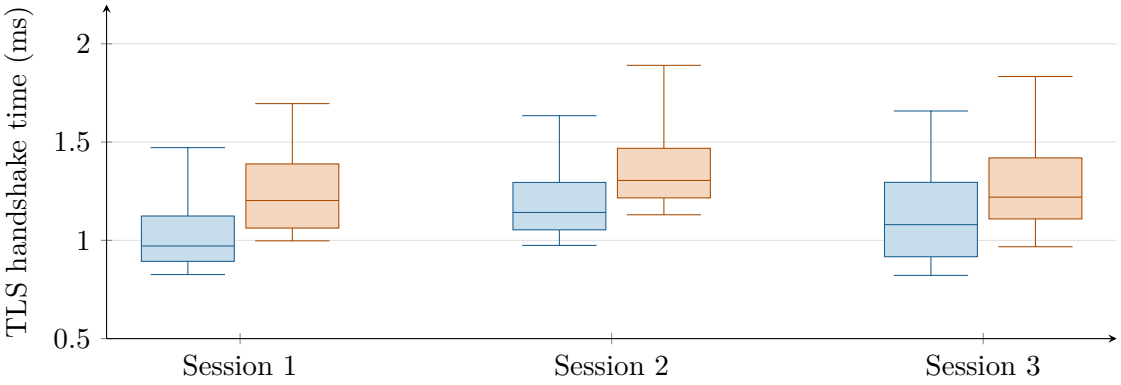



**Figure 3:** Client-measured time of the TLS handshake, **A** (blue) and **B** (orange), 100 per box. Box: quartiles and median; whiskers: 5th and 95th percentile. Connection setup is not included.

| | | TLS handshake (ms, median) | | | Server CPU (ms, median) | | |
|---|---|---|---|---|---|---|---|
| # | Session | **A** | **B** | B/A | **A** | **B** | B/A |
| 1 | s1 | 0.971 | 1.203 | 1.24 | 0.379 | 0.479 | 1.26 |
| 2 | s2b | 1.142 | 1.305 | 1.14 | 0.479 | 0.587 | 1.22 |
| 3 | s3b | 1.080 | 1.220 | 1.13 | 0.377 | 0.465 | 1.23 |

**Table 3:** Medians per session. B/A is the ratio of the two medians.

## 4.2 Processor time on each end

The hybrid handshake costs more processor time on both ends. The client's own CPU time, as measured over the whole connect-and-handshake, rises by a factor of 1.23, 1.17 and 1.17 in the three sessions; the server's by 1.26, 1.22 and 1.23. In absolute terms the added work per handshake is 0.056 to 0.074 ms on the client and 0.088 to 0.107 ms on the server.

## 4.3 Why we quote ratios

The absolute times are not stable between sessions. The server's **A** median is 0.379, 0.479 and 0.377 ms in the three sessions, a spread of about a quarter, although nothing in the software changed and the machines were meant to be idle. The server runs the `powersave` governor and the client is a virtual machine, so processor frequency and host scheduling are plausible causes; we did not test which. The ratio of **B** to **A**, which both configurations share within a session, moves far less (Figure 5).

## 4.4 Bytes and memory

Message sizes and memory are deterministic, not statistical: they were the same in all three sessions. Both ends agree on the bytes in every handshake (what one sent, the other received).

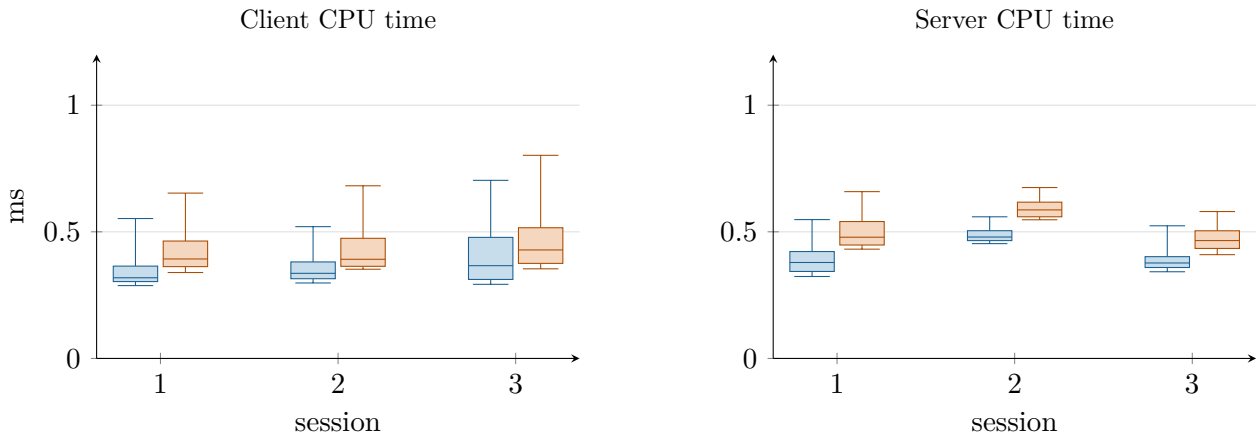



**Figure 4:** CPU time of one handshake on each end, **A** (blue) and **B** (orange). The server runs the slower processor and shows the larger absolute cost.

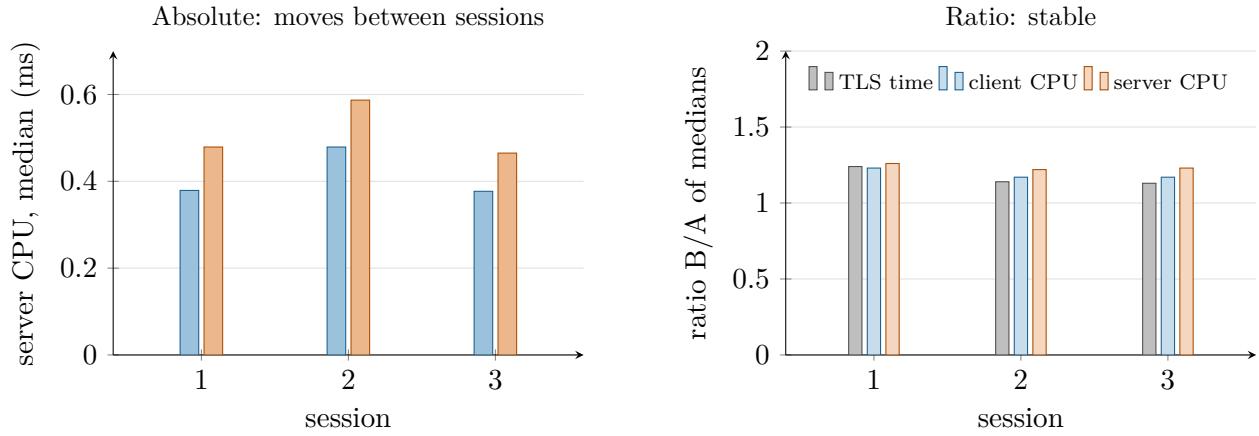



**Figure 5:** Left: the server's CPU time moves by a quarter from session to session (**A** blue, **B** orange). Right: the ratio of **B** to **A** for the handshake time, client CPU and server CPU.

| | A | B | Difference |
|---|---|---|---|
| Client sends in the handshake | 273 | 1,449 | +1,176 |
| Client receives | 757 | 1,845 | +1,088 |
| Heap held by an open connection, client | 19,520 | 27,408 | +7,888 |
| Heap held by an open connection, server | 15,520 | 21,584 | +6,064 |

**Table 4:** Per handshake, median. The byte differences equal those measured on loopback (Report 2 and `results.md`). The client's extra 1,176 bytes are 8 fewer than the 1,184-byte key it sends; we have not explained the 8.

## 5 The slow connects in configuration B

Figure 6 plots every measured attempt. Almost all sit within a few milliseconds. But in every session, exactly two attempts of configuration **B** took about one second from `connect()` to the end of the handshake (6 of the 600 **B** attempts, none of **A**).

What the data show: the time was spent *before* the TLS handshake started (2.4 ms at most for the TLS part, 1015 to 1031 ms in total). About one second is the usual wait before a TCP connection attempt is retried after a lost first packet. So a connection attempt to the **B** server's port was lost and repeated. It does not move the median or the 95th percentile (2 of 100 is below 5 percent), but it would raise a mean or a maximum.

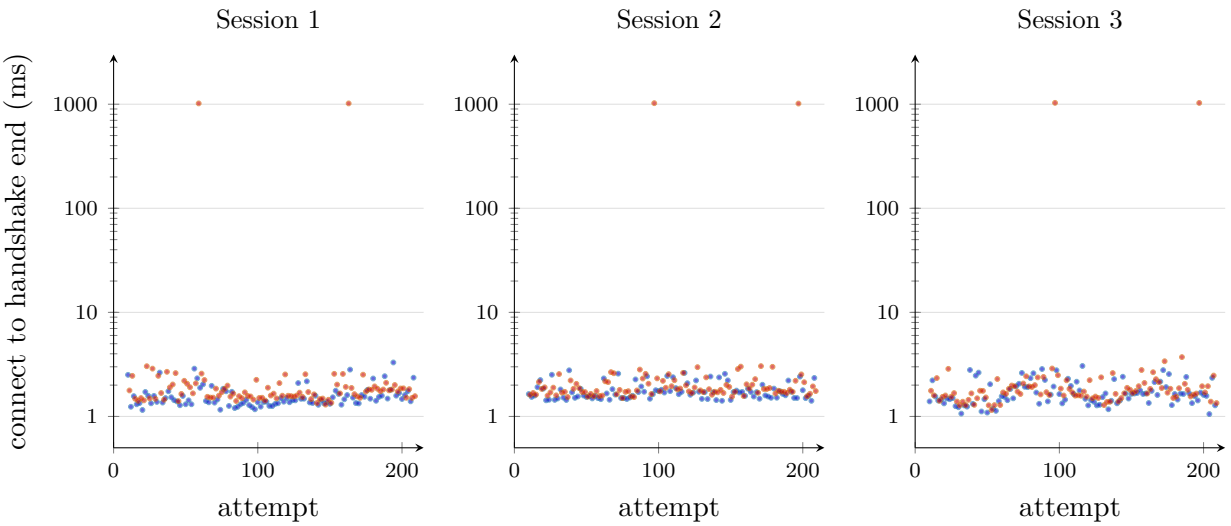



**Figure 6:** Time from `connect()` to the end of the handshake for every measured attempt, on a logarithmic axis (**A** blue, **B** orange). The attempts alternate **A**, **B**, so the two colours interleave along the horizontal axis; the warm-up attempts are not plotted.

| Session | Config | Attempt | Connect to end (ms) | TLS part (ms) |
|---|---|---|---|---|
| 1 | B | 59 | 1018 | 2.4 |
| 1 | B | 163 | 1018 | 1.7 |
| 2 | B | 97 | 1023 | 2.0 |
| 2 | B | 197 | 1015 | 1.5 |
| 3 | B | 97 | 1031 | 2.2 |
| 3 | B | 197 | 1029 | 2.2 |

**Table 5:** The attempts that took longer than 100 ms. The TLS part of the same attempts was normal.

What we do not know: why it only affects **B**, why it comes twice in every session, and where the packet was lost. We did not capture the traffic, because this run is a proof of concept for the procedure and the real test is on the Raspberry Pis. **This is not evidence that the hybrid group makes connections slower**; the TLS part of those attempts is as fast as the others.

## 6 Two sessions we threw away

The first attempts at sessions 2 and 3 were invalid. The check in Figure 2 caught them: the first stopped agreeing with itself, and the second had nothing to summarise.

The facts: the session-2 server files held 149 and 148 handshakes where the client made 105, and were last written about eight minutes after the client finished, so some other client had connected to servers that should have been stopped. A client started under the next identifier found no server and failed all 500 attempts in under four seconds. The later server run recorded no handshakes. Servers left running from a test run were also found on the server machine; they listen on other ports and did not cause this, but they show how easily a forgotten process is left behind.

We moved all of it into a folder named `invalid` with a note, did not edit any file, and reran both sessions under new identifiers. Two changes make this harder to repeat: the server script now refuses to start if its ports are in use, and the runbook says to check for old servers before every session.

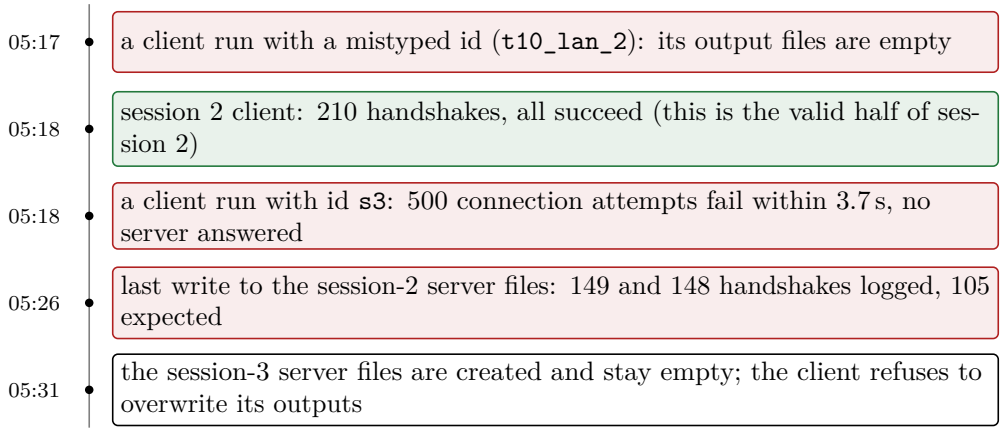



**Figure 7:** What the files show for the two invalid sessions, in order of their time stamps (UTC, minutes past 05:00 on 2026-10-20). Which client run reached the old servers is not recorded.

## 7 What we claim, and what we do not

- **Claimed.** On these two computers and this network, the hybrid handshake (`X25519MLKEM768`) costs about 0.140 to 0.231 ms more than the classical one (`X25519`) and about 1.2 times the processor time on each end. It sends 1,176 bytes more and receives 1,088 more, and holds about 8 kB (client) and 6 kB (server) more memory per connection. The three sessions agree on this.
- **Not claimed.** These are not Raspberry Pi numbers and not gate evidence. The client is a virtual machine. The network is a wired LAN with a sub-millisecond round trip and no loss. Three sessions on one pair of machines show the result repeats here; they are not a statistical statement about other hardware. The one-second connects are unexplained.
- **Authentication.** The certificates are classical (ECDSA P-256). The hybrid key establishment protects the session key against a future quantum computer; it is not post-quantum authentication. The link is an educational one, not flight-approved.

## 8 What comes next

- The same procedure on the two Raspberry Pis, with board identifiers and the instructor's initials; that is the evidence for T10.
- The packet-loss test (T08) across a real network, which needs a way to impose loss on a real interface. The current runner works only inside a private network namespace on one machine.
- Finish `results.md`, the advisor summary and the open questions (queue size, where loss is imposed on the Pis, the role of the Pico).
- The runbook, `two_machine_setup.md`, lists every command used here.